

The background is a dark blue gradient filled with intricate white and light blue circuit-like patterns. These patterns consist of thin lines, dots, and larger circular nodes, some of which are connected by lines, resembling a complex network or a stylized representation of a brain or a digital landscape. In the center, there is a large, glowing, semi-transparent orb or sphere that serves as a focal point. Overlaid on this central orb is a white rectangular box containing the text.

KI-Agenten & smolagents

LLM chain vs Agentic AI

LLM Chain (LLM-Kette)

- Du gibst einen Prompt (z. B. „Fasse diesen Text zusammen“)
- Das LLM gibt eine Antwort zurück
- Kein Denken, keine Werkzeuge – nur Eingabe → Ausgabe
- Gut für: Zusammenfassungen, Textumformulierung, Klassifikation

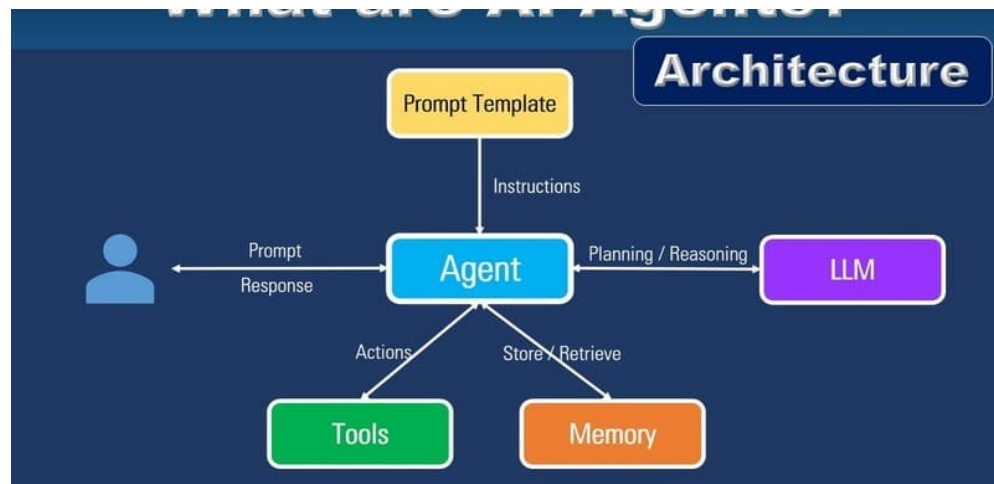
Agent: ein „denkendes“ System mit Werkzeugen:

- Er analysiert dein Anliegen
- Plant Schritte
- Nutzt Tools, z. B. Websuche, Taschenrechner, Dateizugriff
- Gut für: komplexe Aufgaben, Websuche, Rechnen, externe APIs

Merkmal	LLM Chain	Agent
Werkzeugnutzung	✗ Nein	✓ Ja
Logik/Planung	✗ Keine	✓ Eigene Entscheidungen
Komplexität	● Einfach	● Mittel/Komplex
Anwendungsfall	Zusammenfassen, Text	Recherche, Tool-Nutzung

smolagents

- **smolagents** = Nachfolger von transformers.agents
- Agenten mit minimaler Komplexität, klarem Code und starker Unterstützung für Code-Aktionen
- **CodeAgent** als zentrales Feature → Agenten schreiben Aktionen in echtem Code, nicht nur JSON
- **ToolCallingAgent** unterstützt → Für klassische JSON/Text-basierte Aktionen
- **Werkzeuge (Tools)** können vom Hub geladen werden und auf den Hub hochgeladen werden
- Open-Source-Modelle über Hugging Face Inference API, kommerzielle oder lokale Modelle über LiteLLMModel



Multi-Step-Agenten

Instruction: Determine the most cost-effective country to purchase the smartphone model "CodeAct 1". The countries to consider are the USA, Japan, Germany, and India.

Available APIs

[1] lookup_rates(country: str) -> (float, float)

[2] convert_and_tax(price: float, exchange_rate: float, tax_rate: float) -> float

[3] estimate_final_price(converted_price: float, shipping_cost: float) -> float

[4] lookup_phone_price(model: str, country: str) -> float

[5] estimate_shipping_cost(destination_country: str) -> float

LLM Agent using [Text/JSON] as Action

 **Think** I should calculate the phone price in USD for each country, then find the most cost-effective country.

 **Action** **Text:** lookup_rates, Germany
JSON: {"tool": "lookup_rates", "country": "Germany"}

 **Environment** 1.1, 0.19

 **Action** **Text:** lookup_phone_price, CodeAct 1, Germany
JSON: {"tool": "lookup_phone_price", "model": "CodeAct 1", "country": "Germany"}

 **Environment** 700

 **Action** **Text:** convert_and_tax, 700, 1.1, 0.19
JSON: {"tool": "convert_and_tax", "price": 700, "exchange_rate": 1.1, "tax_rate": 0.19}

 **Environment** 916.3

[... interactions omitted (look up shipping cost and calculate final price) ...]

 **Action** **Text:** lookup_rates, Japan
JSON: {"tool": "lookup_rates", "country": "Japan"}

[... interactions omitted (calculate final price for all other countries) ...]

 **Response** The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Fewer Actions Required!

CodeAct: LLM Agent using [Code] as Action

 **Think** I should calculate the phone price in USD for each country, then find the most cost-effective country.

 **Action**

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}

for country in countries:
    exchange_rate, tax_rate = lookup_rates(country)
    local_price = lookup_phone_price("xAct 1", country)
    converted_price = convert_and_tax(
        local_price, exchange_rate, tax_rate
    )
    shipping_cost = estimate_shipping_cost(country)
    final_price = estimate_final_price(converted_price, shipping_cost)
    final_prices[country] = final_price

most_cost_effective_country = min(final_prices, key=final_prices.get)
most_cost_effective_price = final_prices[most_cost_effective_country]
print(most_cost_effective_country, most_cost_effective_price)
```

Control & Data Flow of Code
Simplifies Complex Operations

Re-use 'min' Function from Existing
Software Infrastructures (Python library)

 **Environment** 1.1, 0.19

 **Response** The most cost-effective country to purchase the smartphone model is Japan with price 904.00 in USD.

Multi-Step-Agenten

Code Agent

- Erzeugt Tool-Aufrufe direkt als Python-Code-Snippets; optimiert für Code-Logik und direkte Antworten
- Ermöglicht komplexe Logik: Bedingungen, Schleifen, Kombination mehrerer Tools
- Weniger Rechenschritte, geringer Tokenverbrauch
- Es können Syntaxfehler, Laufzeitfehler oder Ausnahmen auftreten
- Wenn komplexes Reasoning, Ketten und flexible Komposition benötigt werden
- Für Agenten, die als Problemlöser oder Programmierer agieren sollen

Tool-Calling Agent

- Tools werden über ein JSON-Schema beschrieben: Name, Beschreibung, Parameter-Typen usw.
- Geringere Wahrscheinlichkeit für Halluzinationen oder unerwartete Ausgaben.
- Viele interne Schritte, sehr hoher Tokenverbrauch.
- Schwer, komplexe Logik, Kombinationen, Transformationen oder Schleifen abzubilden.
- Agent kann keine eigenen Codestrukturen erstellen. Limitiert auf vordefinierte Aktionen/Tools.
- Wenn transparente Protokollierung wichtig ist, wenn einzelne, atomare Aktionen (z. B. eine API-Abfrage) ausgeführt werden sollen